



Spotify Cloud Analytics: Insights Dashboard

APAN 5450 Term Project

Group 3:

Garima Gupta, Jui Chen Pu, Mika Wada, Tanya Singh, Wenzheng Li



APAN 5400

Cloud Computing



0:23

-3:25

1. Background & Definition

Objective

To develop a **cloud-based analytics system** that processes and visualizes Spotify chart data. The goal is to provide real-time and historical insights into music trends by region, genre, and artist, empowering stakeholders to make informed decisions quickly and effectively.

Business Use Case

- A record label compares artist popularity across regions to plan tours
- Marketing team identifies which music genres trend seasonally
- An independent artist uses the dashboard to track their chart movement relative to competitors

Current Challenges

- Data overload
- Manual limitations
- Lack of real-time insights
- Geographic complexity

Target Audience

- Independent artists
- Music marketers
- Music analysts & researchers
- Spotify internal teams

2. Solution, Data Source & Procurement

Solution

A **cloud-native analytics dashboard** built on AWS, capable of ingesting, storing, analyzing, and visualizing Spotify chart data at scale.

Data Source

Data Source: [Kaggle - Spotify Charts](#)

Dataset Size: 3.48 GB

Data Type: Historical Spotify Chart data including ranking position, song title, artist name, stream count, and region

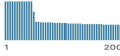
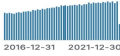
Content: Data of "Top 200" and "Viral 50" charts published globally across 70 regions by Spotify, every 2-3 days. The data spans from January 1, 2017 to December 31, 2021.

Procurement

Procurement: Publicly available datasets

Format: CSV files

Storage: AWS S3 for raw and processed data

▲ title str	# rank int	📅 date date	▲ artist str	📊 uri str	🌐 region category	▲ chart category	▲ trend category	# streams int
164807 unique values			Ed Sheeran 1% Billie Eilish 1% Other (25533772) 98%	217704 unique values		top200 78% viral50 22%	MOVE_DOWN 43% MOVE_UP 37% Other (5152032) 20%	1001 19.7m
Chantaje (feat. Maluma)	1	2017-01-01	Shakira	https://open.spotify.com/track/6e8IAuAdrwEjh6Y61roV2Kg	Argentina	top200	SAME_POSITION	253019
Vente Pa' Ca (feat. Maluma)	2	2017-01-01	Ricky Martin	https://open.spotify.com/track/7DM4BPas7uofFull3yeh4e	Argentina	top200	MOVE_UP	223988
Reggaetón Lento (Bailamos)	3	2017-01-01	CNCO	https://open.spotify.com/track/3AEZUABDXNtecA0SC1qTfo	Argentina	top200	MOVE_DOWN	210943
Safari	4	2017-01-01	J Balvin, Pharrell Williams, BIA, Sky	https://open.spotify.com/track/rv0SvBHT7H1ZjtcM24S4b0	Argentina	top200	SAME_POSITION	173865

3. System Design: Key AWS Services Used

Component	AWS Service	Purpose
Storage	Amazon S3	Store raw data (CSV) from Spotify
Scheduling	AWS EventBridge	Trigger automated Lambda executions on schedule
Compute	AWS Lambda	Perform scheduled tasks, process requests, scan/filter DynamoDB
Database	Amazon DynamoDB & RDS	Structured data storage for dashboard
Compute	Amazon EC2	ETL tasks (database) + Host Streamlit app for dashboard visualization (display)

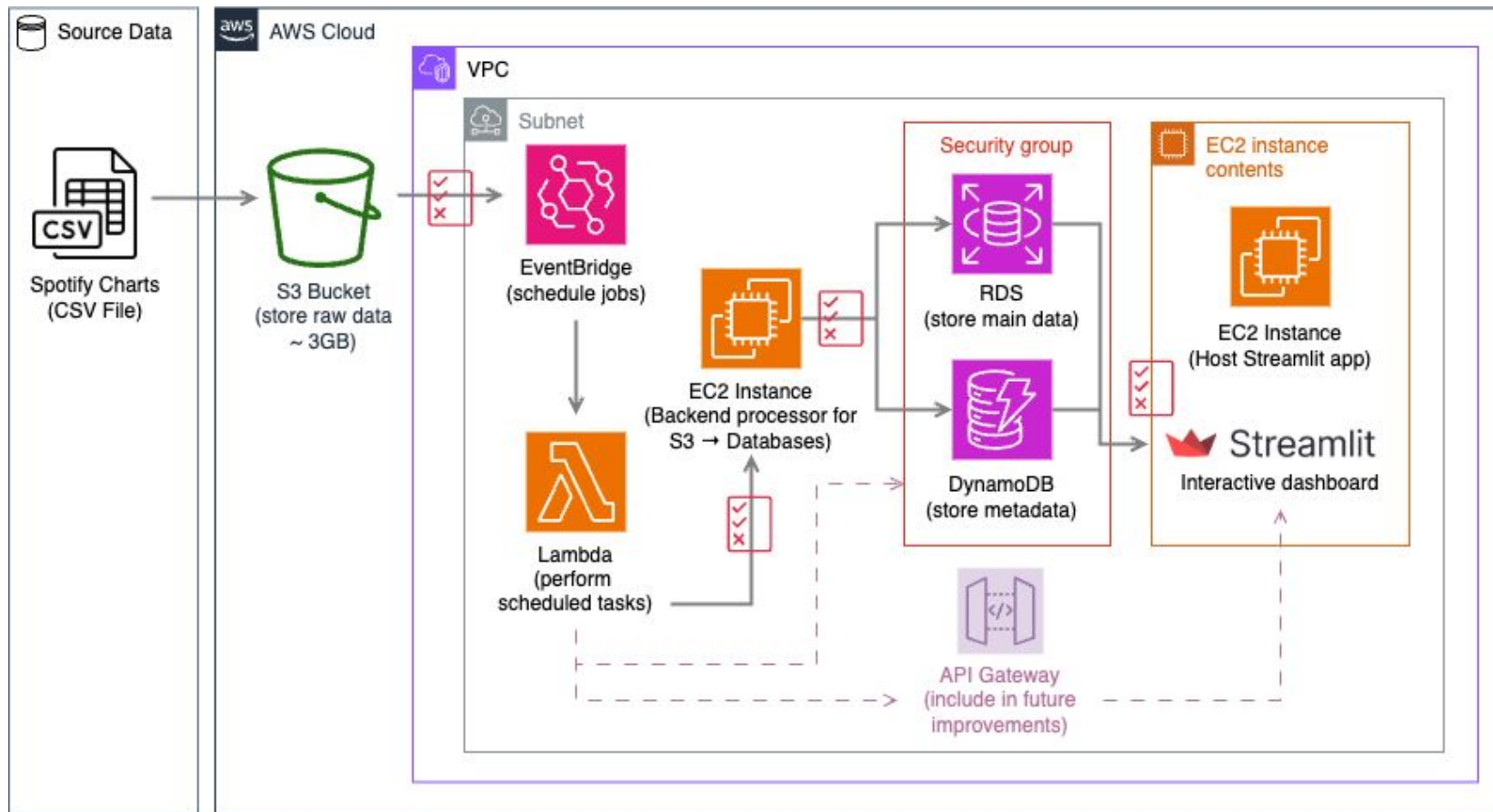


3. System Design: Other Services Used

Component	Service	Purpose
Networking	AWS VPC	Secure and isolate networking environment for RDS and EC2 instances
Security	AWS IAM	Manage access permissions for AWS services and resources
Visualization	Streamlit	Build an interactive web dashboard to visualize music trends



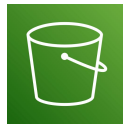
4. High-Level Cloud Architecture



5. Data Storage: S3

Goals

We use Amazon S3 as the **primary storage layer** to handle both **raw and processed Spotify chart data**. It provides highly scalable, durable, and cost-effective object storage, ideal for large datasets like our 3.48GB of CSV/JSON files.



Steps:

1. Create an S3 bucket
2. Upload raw, unprocessed CSV files
3. Organize files into folders
4. Configure access permissions to ensure only authorized services can read the files
5. Use S3 as the central storage point for both raw and processed data

spotify-group3 [Info](#)

[Objects](#)

Metadata

Properties

Permissions

Metrics

Management

Access Points

Objects (4)

[Refresh](#) [Copy S3 URI](#) [Copy URL](#) [Download](#) [Open](#) [Delete](#) [Actions](#) [Create folder](#) [Upload](#)

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

6. Processing & Backend: Lambda

Goals

Lambda function acts as an **event-driven processor that initiates backend workflows** to transform raw Spotify chart data stored in S3 into structured formats within RDS and DynamoDB. This enables scalable and automated preparation of analytics-ready data for our dashboard

Steps:

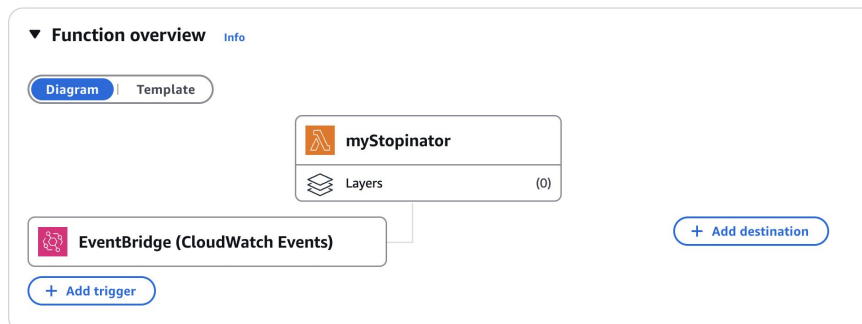
1. Create Lambda functions for each task
2. Set up EventBridge triggers to automate Lambda executions at scheduled intervals
3. Trigger processing jobs on the backend (spotify_database EC2 instance)
4. Extract and transform raw CSV data from S3
5. Load structured data into RDS (for ranking data) and DynamoDB (for metadata)
6. Prepare databases for real-time querying by the Streamlit dashboard



start-ec2-on-s3-upload



myStopinator



7. Database: RDS & DynamoDB

Goals

Store processed Spotify chart data efficiently using a combination of relational (RDS) and NoSQL (DynamoDB) databases to optimize data querying, flexibility, and scalability.

RDS Steps:



1. Create RDS instance inside secure VPC
2. Design relational tables to store ranking data
3. Insert processed data into RDS from Lambda & enable secure access for read/query

spotify-db



Summary

DB identifier spotify-db	Status Available	Role Instance	Engine PostgreSQL	Recommendat 4 informative
CPU 3.63%	Class db.t4g.micro	Current activity 0.00 sessions	Region & AZ us-east-1a	

[Connectivity & security](#) | Monitoring | Logs & events | Configuration | Maintenance & backups | Data migrati

Connectivity & security

Endpoint & port
Endpoint
spotify-db.coqlv9bu6y4s.us-east-1.rds.amazonaws.com
Port
5432

Networking
Availability Zone
us-east-1a
VPC
spotify_vpc-vpc (vpc-0bf301ff3174b5c50)
Subnet group

Security
VPC security groups
DB Security Group (sg-0079856c205007f0c)
Active
Publicly accessible
No

DynamoDB Steps:



1. Create DynamoDB table to store metadata
2. Link records via "meta_id" to connect with RDS
3. Allow Lambda functions to scan and filter entries based on user input from Streamlit app

spotify_metadata



Explore table items

[Settings](#) | Indexes | Monitor | Global tables | Backups | Exports and streams | Permissions

General information info

Partition key
meta_id (Number)

Alarms
No active alarms

Average item size
108.23 bytes

Amazon Resource Name (ARN)
arn:aws:dynamodb:us-east-1:785899696219:table/spotify_metadata

Sort key
-

Point-in-time recovery (PITR)
On

Resource-based policy
Not active

Capacity mode
On-demand

Item count
218,066

Table status
Active

Table size
23.6 megabytes

Get live item count

Additional info

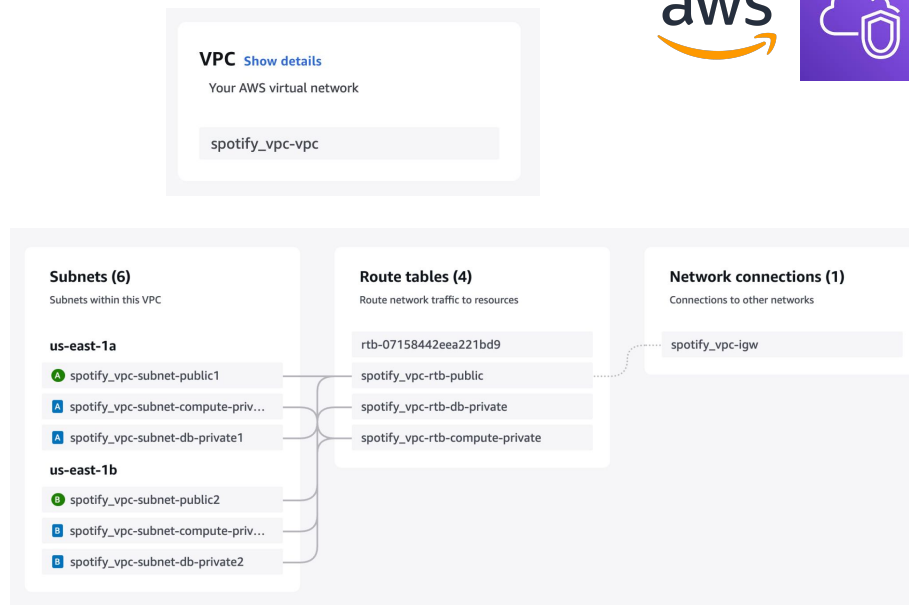
8. Private, Secure Networking: VPC

Goals

The goal of the VPC setup was to create a **secure, isolated network to host backend AWS resources** like EC2, RDS, and DynamoDB. By separating public-facing services from private components across multiple availability zones, we improved both security and scalability. The VPC also allowed controlled traffic flow between services, minimizing exposure to the public internet.

Steps:

1. Created a VPC with IPv4 CIDR 10.0.0.0/16 and enabled DNS resolution and hostnames.
2. Launched 6 subnets: public subnets for frontend services, and private subnets for compute and database layers.
3. Set up a main public route table connected to an Internet Gateway to allow secure outbound internet access.
4. Created private route tables with no direct internet access.
5. Configured network ACLs and security groups to allow only specific inbound and outbound traffic needed for services like EC2, RDS, and API Gateway.



9. Web App Development: Streamlit

Goals

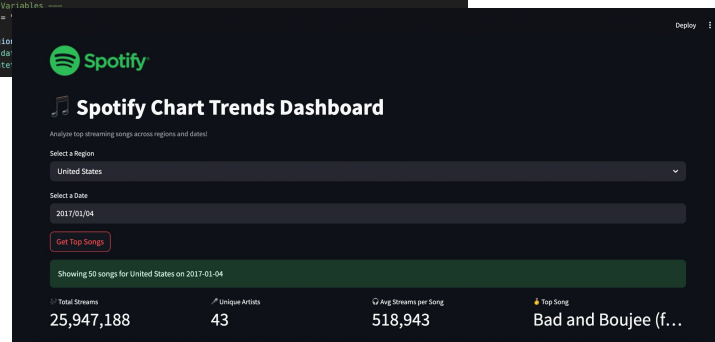
Develop an **interactive dashboard using Streamlit** to visualize Spotify chart trends dynamically based on user-selected region and date, pulling real-time data from RDS and DynamoDB

Steps:

1. Design a Streamlit web app with region and date selection menus (drop-down)
2. Connect to **RDS** to retrieve song rankings and streaming counts based on user input
3. Query **DynamoDB** to fetch detailed song metadata (title, artist) using "meta_id"
4. Merge datasets into a complete view for visualization and analysis
5. Display key statistics (total streams, unique artists, average streams per song, top song)
6. Create interactive charts using **Plotly**
7. Prepare to deploy app on EC2 for public access



```
spotify_dashboard.py >
1 import streamlit as st
2 import pandas as pd
3 import plotly.express as px
4 import requests
5 from PIL import Image
6 import datetime
7
8 # --- Page Setup ---
9 st.set_page_config(page_title="Spotify Chart Trends", layout="wide", initial_sidebar_state="collapsed")
10
11 # --- Logo ---
12 logo = Image.open('spotify.png')
13 st.image(logo, width=150)
14
15 # --- Tabs ---
16 tab1, tab2 = st.tabs(["Dashboard", "API Info"])
17
18 # --- Define Variables ---
19 api_base_url = '
20
21 available_region
22 start_date = da
23 end_date = date:
```



10. Web App Hosting: EC2



Goals

Leverage Amazon EC2 to host both **backend processing services and user-facing dashboard**, ensuring scalable, flexible, and independent operations for data ingestion and visualization

Spotify Database (Backend EC2):

1. Launch EC2 instance (t3.large)
2. Connect to S3 to retrieve raw data (CSV)
3. Process and load structured data into RDS (ranking data) and DynamoDB (metadata).
4. Support data availability for real-time querying by frontend applications

Instances (1/2) info						
Last updated about 1 hour ago						
Find Instance by attribute or tag (case-sensitive)						
All states						
Instance state = running						
Clear filters						
Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
spotify_database	i-03d01169325befe16	Running	t3.large	3/3 checks passed	View alarms	us-east-1a
spotify_display	i-0105fee80bae35a1e	Running	t3.large	3/3 checks passed	View alarms	us-east-1b

i-03d01169325befe16 (spotify_database)			
Instance ID	i-03d01169325befe16		
Public IPv4 address	3.85.40.43 open address		
Private IPv4 addresses	10.0.0.220		
Instance state	Running		
Public IPv4 DNS	ec2-3-85-40-43.compute-1.amazonaws.com open address		
Private IP DNS name (IPv4 only)	ip-10-0-0-220.ec2.internal		
Instance type	t3.large		
Answer private resource DNS name			

Spotify Display (Frontend EC2):

1. Launch EC2 instance (t3.large)
2. Connect directly to RDS and DynamoDB to query prepared data
3. Provide a web interface (via Streamlit)
4. Publicly expose the Spotify dashboard via IP address for user access

Instances (1/2) info						
Last updated about 1 hour ago						
Find Instance by attribute or tag (case-sensitive)						
All states						
Instance state = running						
Clear filters						
Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
spotify_database	i-03d01169325befe16	Running	t3.large	3/3 checks passed	View alarms	us-east-1a
spotify_display	i-0105fee80bae35a1e	Running	t3.large	3/3 checks passed	View alarms	us-east-1b

i-0105fee80bae35a1e (spotify_display)			
Instance ID	i-0105fee80bae35a1e		
Public IPv4 address	44.222.153.125 open address		
Private IPv4 addresses	10.0.2.49		
Instance state	Running		
Public IPv4 DNS	ec2-44-222-153-125.compute-1.amazonaws.com open address		
Private IP DNS name (IPv4 only)	ip-10-0-2-49.ec2.internal		
Instance type	t3.large		
Elastic IP addresses			

11. Security Configurations Overview

Account & Access Management

- IAM roles & permissions were configured to grant AWS services the minimum necessary access
- Strong password policies were configured for account and access to certain services, e.g., databases



Data Security

- S3 bucket permissions were configured to allow access only to authorized Lambda functions
- DynamoDB tables were configured with appropriate IAM policies to restrict access



Networking Security

- VPC was used to secure RDS, DynamoDB, and EC2 instances inside private subnets
- Security groups configured for EC2 instances to allow appropriate access
- S3 bucket access policies enforced to prevent public access



API Security (Future Enhancements)

- API Gateway methods will be configured with IAM-based authorization to control access to the data endpoints
- Throttling and rate-limiting will be enabled at API Gateway level in the future



12. Scalability, Justification, Benefits

Scalability:



AWS Lambda automatically scales with data uploads. S3 provides virtually unlimited storage growth. API Gateway (future improvement) can dynamically handle increasing traffic without downtime.

Security:



IAM roles, VPC security groups, and AWS Shield provide layered security by managing access, securing the network, and defending against cyberattacks.

Efficiency:



Optimized networking through Route 53 and VPC design ensures seamless, low-latency data flow across services.

Performance:



Serverless architecture using Lambda and managed services minimizes operational overhead and optimizes resource costs.

13. Cost Analysis

Cost under current scale			
Server	Cost per Month AWS	Cost per Month GCP	Description
EC2	\$60.74	~\$7.99	t3.large (2 vCPU, 8 GiB RAM), shared tenancy, on-demand, basic web app hosting
S3	\$0.80	~\$0.11	4.48GB storage, basic S3 operations
DynamoDB	\$7.38	~\$4.85	Standard table, on-demand capacity
Lambda	\$0.00	\$0.03	ETL jobs, Free Tier
API Gateway (REST API)	\$3.50	\$0.00	1M requests per month
RDS (PostgreSQL)	\$16.39	\$56.44	db.t3.micro, 20GB SSD, Single-AZ
Total	\$38.14		

Cost Projection (Optional Future Scale)		
Server	Cost per Month	Description
EC2 (m5.xlarge)	~\$93.33	m5.xlarge (4 vCPU, 16 GiB RAM), reserved pricing, for higher load
S3 (Storage growth)	~\$2.00	Additional datasets
DynamoDB (Scaling)	~\$15-20	Higher traffic load
API Gateway (Scaling)	~\$5-8	Increased API calls
RDS (Larger DB)	~\$30-35	More concurrent users
Total Projection	~\$85-105	

14. Team Member Contributions

Member	Responsibilities
Tanya Singh	• Upload raw data into an S3 bucket • Perform preliminary cost analysis
Jui Chen Pu	• Clean and process original dataset using Flask • Build serverless schedule engine using EventBridge and Lambda
Wenzheng Li	• Use RDS to store ranking and time period data in a structured format • Use DynamoDB to store metadata of songs
Garima Gupta	• Build Lambda function to scan and filter DynamoDB based on API input • Create a REST API called with two key endpoints
Mika Wada	• Develop a dashboard using Streamlit for visualizing music trends • Launch an EC2 instance to host the Streamlit app & make it accessible

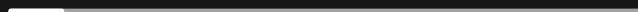
Live Demo



APAN 5450
Cloud Computing



0:23



-3:25



Spotify Chart Trends Dashboard

Analyze top streaming songs across regions and dates!

Select a Region

United States

Select a Date

2017/01/01

Get Top Songs

User selects region and date from drop-down menu
and clicks on "Get Top Songs"



 Total Streams

55,139,058

 Unique Artists

180

 Avg Streams per Song

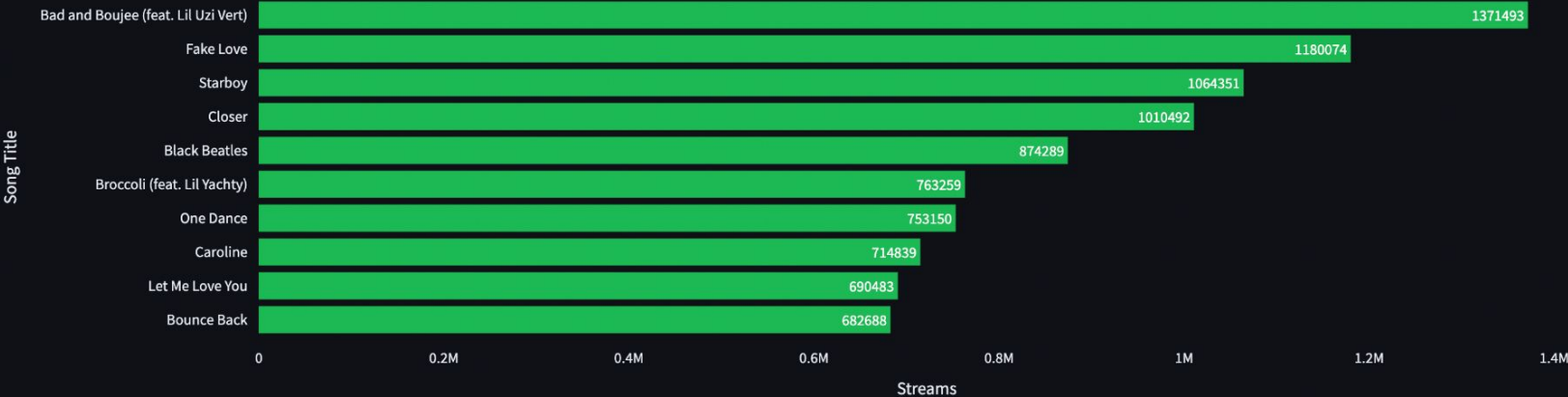
220,556

 Top Song

Bad and Boujee (f...

Top 10 Songs by Streams

Top 10 Streamed Songs





Top 10 Artists by Total Streams



Top 10 Artists by Streams



Application also displays raw song data

Raw Song Data

[illegible]

Thank You!



APAN 5450
Cloud Computing



0:23

-3:25